

## Trabalho Prático – Projeto de Software

Sistema para gestão de oficina de manutenção de eletrodomésticos

|                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>André Felipe de Assis Rodrigues - 2024012099</b><br/><b>Henrique Gustavo Ferreira Silva - 2022066027</b><br/><b>Pedro Henrique E. Dalla-Lana - 2020420613</b><br/><b>Ulisses Aurino Batista Frazão - 2020022715</b><br/><b>Victor Felipe da Cruz Oliveira - 2022035687</b></p> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

Java 17 • Spring Boot 4.0.6 • PostgreSQL 17 • Docker • JUnit 5 • Mockito  
Belo Horizonte — 2026

### 1. Introdução

O projeto Eletropeças Jacaré é uma API RESTful desenvolvida em Java com Spring Boot para gerenciar o ciclo operacional de uma assistência técnica de eletrodomésticos. O sistema abrange quatro domínios principais: clientes (User), fabricantes (Manufacturer), peças em estoque (Part) e ordens de serviço (Order). O foco desta documentação está na decisões de implementação e na estratégia de testes adotada.

### 2. Arquitetura e Estrutura do Projeto

O projeto adota uma arquitetura em camadas inspirada nos princípios do Domain-Driven Design (DDD). Cada módulo de negócio é subdividido em quatro pacotes verticais com responsabilidades bem delimitadas.

O pacote Domain contém a classe de domínio pura, sem anotações de framework. É nele que residem as regras de negócio mais fundamentais — por exemplo, a validação de e-mail no construtor de User e o método hasAvailableStock() em Part. O pacote Application contém a interface de serviço e sua implementação, que orquestra os casos de uso. O pacote Persistence reúne a entidade JPA, o repositório Spring Data e o Mapper responsável por converter entre entidade e domínio. Por fim, o pacote Api expõe os endpoints REST e os DTOs de resposta, implementados como Java Records.

A estrutura de pacotes completa segue o padrão:

```

com.jacare.eletopecas/
  EletopecasApplication.java
  Manufacturer/ | Order/ | Part/ | User/
    Api/
      Dto/*Response.java    <- Java Record (DTO de saída)
      *Controller.java      <- @RestController
    Application/
      I*Service.java        <- interface do contrato
      *Service.java         <- @Service com a lógica de negócio
    Domain/
      *.java                <- classe de domínio pura
    Persistence/
      *Entity.java          <- @Entity JPA
      *Mapper.java          <- conversão domínio <-> entidade
      *Repository.java      <- extends JpaRepository

```

Essa organização evita o vazamento de anotações de infraestrutura para o domínio e facilita a testabilidade, pois a lógica de negócio pode ser testada sem carregar o contexto Spring.

### 3. Implementação

#### 3.1 Módulo User

A classe de domínio User é a única no projeto que possui regra de negócio no próprio construtor: caso o e-mail seja nulo ou não contenha '@', uma IllegalArgumentException é lançada imediatamente, antes mesmo de qualquer camada de persistência ser envolvida.

```

public User(Long id, String name, String email, String cpf, String passwordHash) {
    if (email == null || !email.contains("@"))
        throw new IllegalArgumentException("O e-mail fornecido é inválido.");
    // atribuições...
}

```

Na camada de persistência, UserEntity declara os campos email e cpf com @Column(unique = true), garantindo unicidade no banco. O UserService traz uma regra importante na operação de atualização: a senha só é substituída se um valor não nulo e não vazio for fornecido, evitando sobrescrita acidental:

```
if (updatedUser.getPasswordHash() != null &&
!updatedUser.getPasswordHash().isBlank()) {
    existingUser.setPasswordHash(updatedUser.getPasswordHash());
}
```

O DTO de resposta `UserResponse` omite intencionalmente o campo `passwordHash`, garantindo que o hash nunca seja exposto pela API.

### 3.2 Módulo Part — Controle de Estoque

A classe `Part` contém o método `hasAvailableStock(int amount)`, que encapsula a regra de verificação de estoque no domínio:

```
public boolean hasAvailableStock(int amount) {
    return this.quantityInStock >= amount;
}
```

O `PartService` implementa três operações além do CRUD padrão. O método `increaseStock()` valida que a quantidade é positiva antes de incrementar. O método `decreaseStock()` combina essa validação com a checagem de `hasAvailableStock()` — se o estoque for insuficiente, lança `IllegalStateException`. O método `getPartsBelowStock()` filtra em memória as peças cuja quantidade está abaixo de um limiar mínimo, retornando uma lista útil para alertas de reposição:

```
public void decreaseStock(Long partId, int quantity) {
    if (quantity <= 0)
        throw new IllegalArgumentException("A quantidade deve ser maior que zero");
    Part part = getPartById(partId);
    if (!part.hasAvailableStock(quantity))
        throw new IllegalStateException("Estoque insuficiente para a peça " + partId);
    part.setQuantityInStock(part.getQuantityInStock() - quantity);
    partRepository.save(PartMapper.toEntity(part));
}
```

### 3.3 Módulo Order — Ciclo de Vida

O módulo `Order` é o mais complexo do projeto. Uma ordem de serviço agrega dados do eletrodoméstico (descrição, marca, número de série, defeito relatado), o cliente responsável, uma lista de peças necessárias para o reparo, custos financeiros e prazos.

O ciclo de vida é modelado pelo enum `OrderStatus` com seis estados. Uma OS nasce sempre em `BUDGET_PENDING` e evolui pelas transições abaixo conforme as ações do técnico ou do cliente:

|                        |                                                             |
|------------------------|-------------------------------------------------------------|
| <b>BUDGET_PENDING</b>  | Estado inicial. Aguardando análise e orçamento.             |
| <b>BUDGET_APPROVED</b> | Cliente aprovou o orçamento.                                |
| <b>IN_REPAIR</b>       | Aparelho distribuído ao técnico.                            |
| <b>READY</b>           | Reparo concluído, aguardando retirada.                      |
| <b>DELIVERED</b>       | Aparelho entregue — fim do ciclo positivo.                  |
| <b>CANCELLED</b>       | Orçamento recusado ou OS cancelada — fim do ciclo negativo. |

Cada transição é implementada como um método dedicado no `OrderService`, como `approveBudget()`, `startRepair()`, `markAsReady()` e `deliverOrder()`. O serviço também oferece dois métodos de cálculo financeiro: `calculateMaterialCost()` soma os preços (`supplierPrice`) de todas as peças associadas, e `calculateTotalCost()` adiciona o custo de mão de obra (`laborCost`) ao resultado.

Na persistência, o relacionamento com o cliente é mapeado como `@ManyToOne`. A lista de peças usa `@ManyToMany` com uma tabela associativa `order_parts` gerada automaticamente pelo JPA. O status é persistido como `String` via `@Enumerated(EnumType.STRING)`, tornando o valor legível diretamente no banco.

### 3.4 Padrão Mapper

Cada módulo possui uma classe `Mapper` estática que isola a conversão entre o objeto de domínio e a entidade JPA. Esse padrão evita que anotações de infraestrutura (como `@Entity` ou `@JoinColumn`) contaminem o domínio, e centraliza a lógica de mapeamento em um único lugar. O exemplo abaixo é do `ManufacturerMapper`:

```
public class ManufacturerMapper {  
    public static Manufacturer toDomain(ManufacturerEntity e) {  
        if (e == null) return null;  
        return new Manufacturer(e.getId(), e.getName(), e.getContactEmail());  
    }  
    public static ManufacturerEntity toEntity(Manufacturer m) {  
        if (m == null) return null;  
        return new ManufacturerEntity(m.getId(), m.getName(), m.getContactEmail());  
    }  
}
```

```
}  
}
```

O OrderMapper trata os casos mais complexos, incluindo a lista de peças e o mapeamento aninhado de UserMapper e PartMapper para recompor os relacionamentos ao converter de entidade para domínio.

### 3.5 DTOs de Resposta como Records

Todos os DTOs de resposta são implementados como Java Records, que são imutáveis por definição. Cada Record expõe um método estático de fábrica toResponse() que recebe o objeto de domínio e produz o DTO. Essa abordagem controla exatamente quais campos são serializados no JSON de resposta — notavelmente, o passwordHash é excluído de UserResponse, e o PartResponse aninha um ManufacturerResponse completo:

```
public record PartResponse(  
    Long id, String description,  
    Integer quantityInStock, Double supplierPrice,  
    ManufacturerResponse manufacturer) {  
  
    public static PartResponse toResponse(Part part) {  
        if (part == null) return null;  
        return new PartResponse(part.getId(), part.getDescription(),  
            part.getQuantityInStock(), part.getSupplierPrice(),  
            ManufacturerResponse.toResponse(part.getManufacturer()));  
    }  
}
```

## 4. Testes Automatizados

### 4.1 Estratégia e Configuração

A suíte de testes é composta por três classes com propósitos distintos: testes de domínio (UserTest), testes de serviço com mock (UserServiceTest) e um teste de contexto (ElektropeçasApplicationTests). Os testes de serviço e o de contexto utilizam o banco H2 em memória, configurado no arquivo src/test/resources/application.properties para operar no modo de compatibilidade com PostgreSQL:

```
spring.datasource.url=jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1;MODE=Postgre  
SQL  
spring.jpa.hibernate.ddl-auto=create-drop
```

A propriedade `DB_CLOSE_DELAY=-1` mantém o banco H2 ativo durante toda a execução da JVM de testes. A estratégia create-drop garante que o schema seja recriado do zero a cada execução, eliminando interferência entre testes.

## 4.2 Testes de Domínio — `UserTest`

A classe `UserTest` verifica o comportamento do construtor de `User` diretamente, sem nenhum mock ou dependência de framework. Os dois casos cobrem o caminho feliz e o caminho de erro da validação de e-mail:

```
@Test
@DisplayName("Deve criar um usuário com dados válidos")
void shouldCreateUserWithValidData() {
    User user = new User(1L, "André Silva", "andre@email.com", "123.456.789-00",
"hash");
    assertAll(
        () -> assertEquals("André Silva", user.getName()),
        () -> assertEquals("andre@email.com", user.getEmail()),
        () -> assertEquals("123.456.789-00", user.getCpf())
    );
}
```

```
@Test
@DisplayName("Deve falhar se tentar criar usuário com e-mail inválido")
void shouldThrowExceptionWhenEmailIsInvalid() {
    assertThrows(IllegalArgumentException.class, () ->
        new User(1L, "André Silva", "123", "123.456.789-00", "senha")
    );
}
```

O uso de `assertAll()` no primeiro caso garante que todas as asserções sejam avaliadas mesmo que uma falhe, produzindo um relatório de erro mais completo. O `assertThrows()` no segundo caso confirma que a regra de validação do construtor de domínio está funcionando corretamente.

## 4.3 Testes de Serviço — `UserServiceTest`

A classe `UserServiceTest` utiliza Mockito para isolar o `UserService` de sua dependência real com o banco de dados. A anotação `@ExtendWith(MockitoExtension.class)` ativa o processamento do Mockito, `@Mock` cria

um substituto simulado do UserRepository, e @InjectMocks injeta esse substituto no UserService:

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
    @Mock
    private UserRepository userRepository;
    @InjectMocks
    private UserService userService;
}
```

#### Caso 1: Criação de usuário com sucesso

O teste define o comportamento esperado do mock com when...thenReturn, executa o método e verifica o resultado com assertNotNull() e assertEquals(). O verify() ao final confirma que o repositório foi acionado exatamente uma vez:

```
when(userRepository.save(any(UserEntity.class))).thenReturn(userEntity);

User savedUser = userService.createUser(userDomain);

assertNotNull(savedUser);
assertEquals("andre@email.com", savedUser.getEmail());
verify(userRepository, times(1)).save(any(UserEntity.class));
```

#### Caso 2: Senha não deve ser substituída por valor vazio

Este teste valida a regra de negócio de preservação de senha implementada no updateUser(). O mock do findById() retorna uma entidade com a senha original, e o mock do save() devolve a mesma entidade. Ao fornecer uma string vazia para passwordHash na requisição de atualização, o teste confirma que a senha original é mantida:

```
UserEntity existingEntity = new UserEntity(1L, "André", "andre@email.com",
    "123.456.789-00", "$2a$original_hash");
User updateData = new User(1L, "André", "andre@email.com", "123.456.789-00",
    "");

when(userRepository.findById(1L)).thenReturn(Optional.of(existingEntity));
when(userRepository.save(any(UserEntity.class))).thenReturn(existingEntity);

User updatedUser = userService.updateUser(1L, updateData);
```

```
assertEquals("$2a$original_hash", updatedUser.getPasswordHash(),  
    "A senha não deveria ter mudado");
```

#### 4.4 Teste de Contexto — EletropecasApplicationTests

O teste `contextLoads()` é anotado com `@SpringBootTest`, o que instrui o Spring a carregar o contexto de aplicação completo antes de executá-lo. O método não contém asserções: sua simples execução bem-sucedida já constitui a verificação — se qualquer bean estiver mal configurado, a injeção de dependência falhar ou uma propriedade obrigatória estiver ausente, o contexto não sobe e o teste falha. Funciona como uma barreira de sanidade para garantir que a aplicação consegue ser inicializada.

### 5. Infraestrutura

---

#### 5.1 Dockerfile (Multi-Stage Build)

O Dockerfile utiliza build multi-estágio para minimizar a imagem final. O estágio de build usa a imagem `maven:3.9.11-eclipse-temurin-21` para compilar o projeto e gerar o `.jar`. O estágio de runtime usa apenas o `eclipse-temurin:21-jre` — o JRE sem ferramentas de compilação — e copia somente o artefato gerado. O resultado é uma imagem de produção significativamente menor, sem código-fonte ou dependências de build:

```
FROM maven:3.9.11-eclipse-temurin-21 AS build  
WORKDIR /app  
COPY pom.xml . && COPY .mvn .mvn && COPY mvnw .  
RUN chmod +x mvnw && ./mvnw dependency:go-offline  
COPY src src  
RUN ./mvnw clean package -DskipTests  
  
FROM eclipse-temurin:21-jre  
WORKDIR /app  
COPY --from=build /app/target/*.jar app.jar  
EXPOSE 8080  
ENTRYPOINT ["java", "-jar", "app.jar"]
```

#### 5.2 Docker Compose

O `docker-compose.yml` define dois serviços em rede interna. O serviço `postgres` usa a imagem oficial `postgres:17` e persiste dados em um volume nomeado (`postgres-data`). O serviço `api` constrói a imagem a partir do Dockerfile local e recebe as credenciais do banco via variáveis de ambiente, que são injetadas nos placeholders do `application.properties` (`{SPRING_DATASOURCE_URL}`, `{SPRING_DATASOURCE_USERNAME}`),



`}${SPRING_DATASOURCE_PASSWORD})`). A diretiva `depends_on` garante a ordem de inicialização dos containers.

## 6. Considerações Finais

---

O projeto demonstra, de forma coesa, a aplicação de princípios modernos de engenharia de software backend em um contexto prático e relevante. A adoção da arquitetura em camadas com separação clara entre domínio, aplicação, persistência e API resulta em um código com alta manutenibilidade e testabilidade.

A modelagem do ciclo de vida das Ordens de Serviço através do enum `OrderStatus` e dos métodos de transição de estado no serviço reflete uma compreensão adequada do processo de negócio real, traduzindo-o fielmente para o modelo de software. O controle de estoque com validações de domínio (`hasAvailableStock`) demonstra a aplicação do princípio de responsabilidade única, onde a regra de negócio reside na classe que a representa.

A estratégia de testes adotada, com testes de unidade isolados por meio de mocks (Mockito), testes de domínio diretos e um banco de dados em memória (H2) para o contexto de testes proporcionam uma cobertura de qualidade sem dependências externas, tornando os testes rápidos e confiáveis.

A containerização com Docker e o build multi-estágio demonstram preocupação com a operacionalização do sistema, facilitando a implantação em qualquer ambiente que suporte Docker, seja local, em nuvem pública ou em servidores on-premise.